



NoSQL dans un projet informatique : atouts et lacunes

NFE204 – B3D

BASES DE DONNEES DOCUMENTAIRES ET DISTRIBUEES

Résumé

NoSQL devient un incontournable du stockage de données à haute volumétrie. Certains croient même que ces systèmes sont l'avenir des bases de données. Mais seriez-vous prêt à risquer la réussite d'un projet informatique en sautant le pas ? Cette étude vous permettra de mieux connaître les systèmes NoSQL et de les comparer aux SGBDR. Elle apporte une réponse construite sur les risques et les bénéfices envisageable lorsque l'on passe aux bases de données documentaires.

Table des matières

Préambule	1
Connaitre les origines.....	2
Pour comprendre la philosophie	4
L'objectif des SGBDR	4
L'objectif de NoSQL	4
Le choix du système NoSQL est déterminant.....	6
Le choix en fonction des optimisations fonctionnelles	6
Les bases de données orientées document	6
Les bases de données clé-valeur	6
Les bases de données orientées colonnes	7
Les bases de données orientées graphes.....	7
Le choix en fonction du format des données.....	7
Une technologie en pleine effervescence.....	9
Les leaders du relationnel se lancent dans le NoSQL.....	9
Certains proposent de nouveaux produits.....	9
D'autres tentent la fusion des deux concepts.....	9
Tout n'est pas mauvais dans les SGBDR.....	10
SQL : Un langage qui a de l'avenir	10
Séduire les partisans des propriétés ACID.....	11
Le BigData pour tous sur le Cloud	11
Des avis partagés.....	12
NoSQL futur leader.....	12
L'expérience difficile du NoSQL.....	12
Conclusion	14

Préambule

L'engouement du NoSQL amène certains de ses partisans à affirmer que ces nouvelles bases de données pourraient prendre le pas sur les SGBD relationnels.

C'est de ce constat que naquit l'idée du projet, précédemment livré, complété par l'étude ci-présente. L'objectif principal de ces deux rapports est de savoir si nous sommes en droit de douter de cet engouement grandissant ou si nous devrions tout simplement accepter le changement d'ère de la persistance des données numériques.

La première étape était le projet. Au cours de celui-ci, j'ai réalisé une application web dont les données étaient stockées dans une base de données NoSQL. J'ai choisi de créer une architecture pour laquelle les bases de données relationnelles sont parfaitement adaptées. La base mise en place s'est avérée tout aussi efficace. Le chargement des documents en base s'est montré extrêmement simple mais nous avons pu constater que cette simplicité impliquait une contrepartie pour l'application. La complexité liée au contrôle de l'intégrité des données a été reportée dans le code car il a fallu décrire précisément chaque objet manipulé provenant de la base. La conclusion de cette première partie fut donc bonne, dans le sens où les gestionnaires NoSQL semblent aptes à relever efficacement les SGBDR en offrant en prime quelques nouvelles fonctionnalités très pratiques. Une légère amertume vient tout de même nuancer l'enthousiasme que l'on aurait pu avoir si l'on tient compte de l'absence de contraintes d'intégrité des données.

Avec cette seconde étape, je vais tenter d'apporter le maximum d'éléments comparatifs pour que l'on puisse se faire une idée précise de ce qui différencie un système relationnel d'une base de données documentaire. On se rendra alors compte des capacités de chacun et on sera à même de déterminer si ces systèmes sont équivalents, complémentaires, ou si l'un d'eux a des chances d'éclipser l'autre. Cette conclusion sera déterminante pour la réussite de futurs projets informatiques de grande ampleur.

Connaitre les origines

Les gestionnaires de base de données NoSQL sont, par opposition aux SGBDR, tous les autres systèmes de stockage de données n'imposant pas fortement de structure et ne proposant pas de mécanisme de mise en relation des données. En se limitant à cette définition, on se rend compte que le NoSQL n'est pas né d'hier. Un classeur Excel, un annuaire LDAP ou même un gestionnaire de fichier peut être considéré comme une base NoSQL. Cependant, cette appellation est relativement récente car elle serait apparue à l'aube du 21^e siècle et n'a connu un réel essor qu'au début de cette décennie.

<https://fr.wikipedia.org/wiki/NoSQL>

Il y a moins de 10 ans, les géants d'Internet ont commencé à atteindre les limites des bases de données relationnelles. En voici trois exemples.

Le moteur de recherche Google s'est engagé dans la lourde tâche de référencer la totalité du web ouvert. Cela représente une quantité de donnée gargantuesque augmentant au rythme effréné d'une centaine de Go par jour.

<http://www.silicon.fr/base-donnees-nosql-impose-sgbd-93305.html>

Amazon, e-commerçant à influence mondiale, a connu son succès grâce à la vente de livre, CD et DVD. Il décide alors d'étendre son marché à des secteurs tels que l'animalerie, les véhicules personnels, les bijoux, les vêtements, le jardin... Pour devenir incontournable et proposer toujours plus de produits, il ouvre sa boutique aux internautes. Tout un chacun, professionnel ou particulier, peut venir gonfler le catalogue en y enregistrant ses articles. Amazon compte maintenant plus de 183 millions de références et enregistre presque 500 transactions par seconde pendant les périodes de fêtes.

<http://www.theverge.com/2013/12/26/5245008/amazon-sees-prime-spike-in-2013-holiday-season>

<http://mashable.com/2012/12/16/how-big-is-amazon/>

Facebook, plateforme incontournable de la vie sociale électronique, voit son nombre d'utilisateur augmenter tous les jours sans le moindre répit. Au nombre de 500 millions en 2010, il en compte aujourd'hui presque trois fois plus et environ 3 milliards de partages. Ce volume aurait pu être acceptable si la disponibilité et la rapidité du site ne faisait pas partie des contraintes principales. Rappelons tout de même que le site doit traiter plus de 13 millions de requêtes par secondes.

<http://www.statista.com/statistics/264810/number-of-monthly-active-facebook-users-worldwide/>

https://ycharts.com/companies/FB/shares_outstanding

Ces trois géants de la toile sont les exemples typiques de la naissance du BigData. Les volumes de données et de requêtes sont si importants que les SGBDR saturent. Pour satisfaire ces nouvelles exigences, chacun d'eux s'est lancé dans le développement de gestionnaires de bases de données moins axé sur l'intégrité des données mais optimisés pour le stockage de volumes hors normes, pour la rapidité d'exécution des requêtes et pour la continuité de service. C'est ainsi que Google mis au point « Big Table ». Se basant sur ces travaux, Amazon entreprit « SimpleDB » et Facebook, « HBase ».

Ces trois entreprises ont comme point commun d'avoir vu le jour grâce à leur site Internet et ont toutes trois été contraintes de développer un gestionnaire de données adapté à leurs besoins. Dans cette quête à l'optimisation du traitement des données, d'autres sites ont emboité le pas. On peut noter par exemple Twitter avec « Cassandra » ou LinkedIn avec « Voldemort ». Le site nosql-database.org référence aujourd'hui presque 200 noms de gestionnaires NoSQL. Sachant que la plupart des éditeurs proposent leur produit sous licence libre, cette concurrence au monde relationnel devient extrêmement attractive pour les deux raisons suivantes :

- les bases de données NoSQL sont optimisées pour le BigData avec des optimisations répondant à des contraintes diverses selon l'éditeur ;
- l'investissement dans ces systèmes peut très bien être nul.

<http://nosql-database.org/>

Pour comprendre la philosophie

Les gestionnaires NoSQL sont performants. Ce constat est vérifiable quotidiennement sur les sites cités précédemment. Mais comment font-ils pour être plus performants que les systèmes relationnels ?

L'objectif des SGBDR

Un gestionnaire de base de données relationnel se doit de garantir l'intégrité des données. Pour cela, il met en place de lourds mécanismes qui lui permettent d'assurer sa propriété dite ACID. Cet acronyme désigne quatre caractéristiques essentielles : Atomicité, Cohérence, Isolation et Durabilité.

L'atomicité garantie qu'un groupe de modifications qui n'ont pas lieu d'être exécutées partiellement seront toutes exécutées correctement ou qu'aucune d'elles ne sera exécutée. Pour cela, le SGBDR propose un mécanisme de transaction. Toutes les requêtes placées dans cette transaction sont exécutées en cache. Dans le cas où une anomalie se produit, la transaction est annulée. Si tout se passe bien, la transaction est validée, ce qui aura pour effet de mettre à jour les données avec ce nouvel état qui était temporaire.

La cohérence garantie que les contraintes d'intégrité mises en place seront respectées en tout temps. Les contraintes sont définies par la mise en place de schémas décrivant les tables et leurs champs. Parmi ces contraintes, on trouve les contraintes par clés :

- clé primaire : la donnée est unique par cette clé ce qui la rend référençable par cette valeur ou ce groupe de valeurs ;
- clé étrangère : référence une donnée par sa clé primaire en garantissant que cette donnée existe.

Et les contraintes de saisie : défini si la donnée peut être nulle, si elle doit être unique, sa valeur par défaut, son type, ses limites...

L'isolation garantie qu'une donnée en cours de modification ne sera pas accessible tant qu'elle n'aura pas retrouvé un état cohérent. Pour réaliser l'isolation, le gestionnaire met en place des verrous sur les tables qu'il est en train de manipuler. Il existe deux types de verrous. Certains se limitent à bloquer l'écriture alors que d'autres sont plus restrictifs en bloquant également la lecture.

La durabilité garantie que la donnée modifiée sera persistée, même en cas de panne. Cette donnée ne peut donc pas rester en RAM. Elle doit être écrite sur un support physique pour que la base valide sa transaction.

<http://www.gaudry.be/sgbdr-acid.html>

<http://www.commentcamarche.net/contents/1055-sql-contraintes-d-integrite>

L'objectif de NoSQL

Les propriétés ACID d'une base sont un atout important pour l'intégrité des données. Cependant, pour certains projets informatiques, ces propriétés peuvent s'avérer trop contraignantes aux vues des bénéfices qu'elles apportent. L'idée fut donc de s'affranchir de cette ligne de conduite et de se concentrer sur les performances. Une nouvelle idéologie vit alors le jour : les propriétés CAP ou CPD en français. Elles orientent les actions du gestionnaire de base de données selon les trois contraintes suivantes :

- Consistance : garantie que l'état d'une base sera identique quel que soit le nœud qui traitera la requête ;

- Disponibilité : assure que toute requête donnera lieu à une réponse de la part de la base ;
- Tolérance au partitionnement : la panne d'un élément du système ou la rupture d'une liaison entre deux groupes de nœuds n'empêchera pas le système de répondre correctement.

Ces contraintes sont bien sûr très dépendantes de l'architecture matérielle mise en place mais chaque gestionnaire garantie de pouvoir suivre ces règles s'il est déployé sur un nombre suffisant de serveurs.

On évite alors de perdre du temps de calcul sur les vérifications de contraintes d'intégrité et on fait un maximum d'effort sur la distribution des tâches sur l'ensemble des unités de calculs disponibles.

https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_CAP

Le choix du système NoSQL est déterminant

Les gestionnaires de bases de données NoSQL répondent à un objectif commun : réduire les contrôles d'intégrité pour améliorer les capacités et les performances. Mais, comme nous venons de le voir, chaque éditeur a développé son gestionnaire pour répondre à des contraintes qui lui étaient propres. Google souhaitait stocker un volume considérable de données. Amazon a probablement orienté ses développements sur la sécurité de son système étant donné qu'il stocke des informations de paiement et traite énormément de transaction financières. Quant à Facebook, sa plus grande préoccupation était liée à la disponibilité des données et à la rapidité de traitement des requêtes. Malgré les optimisations communes, il ne serait donc pas étonnant de constater des écarts entre les différents systèmes.

Le choix en fonction des optimisations fonctionnelles

Un premier regroupement évident s'opère lorsque l'on compare la manière dont les données sont organisées et indexées. Certains systèmes ingèrent les documents tels quel. D'autres demandent à lier une clé à chaque entrée. D'autres encore font automatiquement des indexations qui optimisent la recherche plein texte. On trouve même des gestionnaires qui ont conservé l'aspect relationnel (sans les contraintes d'intégrité) des SGBDR. Voici particularités de chacune.

<https://en.wikipedia.org/wiki/NoSQL>

<http://www.lemagit.fr/article/NoSQL-le-choix-difficile-de-la-bonne-technologie>

Les bases de données orientées document

Les serveurs tels que MongoDB et CouchDB font partie de cette catégorie. Les données peuvent avoir des structures diverses, ce qui peut parfois être vu comme un avantage ou un gros inconvénient selon le système informatique à développer. Cette flexibilité dans la structure des données fait que chaque entrée est considérée comme un document.

Ces bases stockent des données sans la moindre contrainte rendant ainsi l'opération d'insertion aussi rapide que cela puisse être. La recherche d'un document précis est cependant nettement plus acrobatique car, une fois inséré, il est noyé dans la masse de documents que contient la base. Ce type de base est donc peu adaptée à la recherche unitaire et, par extension, à la modification ou à la suppression de documents. En revanche, elles se démarquent par leur rapidité d'insertion et par leur performance dans les recherches par similarité.

Les bases de données clé-valeur

On compte Redis, Riak et FoundationDB dans cette catégorie. Ces bases offrent une fonctionnalité intéressante vis-à-vis des bases orientées documents : chaque document est référencé par une clé. Ces bases sont plus efficaces dans la recherche d'un document unique, si tant est que l'on connaisse sa clé. Cela compense la faiblesse des bases orientées document dans la modification ou la suppression de documents et offre une très haute performance dans la recherche de document par clé.

Cette particularité peut aussi est perçue comme une contrainte car la gestion des clés et de leur unicité en revient à l'utilisateur ou à l'application qui insérera les documents. Une insertion avec une clé existante revient à écraser le document qui utilisait cette clé et la difficulté réside alors en la génération d'une clé représentant le document de manière unique et bidirectionnelle.

Les bases de données orientées colonnes

On compte dans cette catégorie les gestionnaires des géants du net : Cassandra ainsi que BigTable, sur lequel ont été fondés SimpleDB et HBase. Le concept de ces bases est fondamentalement différent d'un stockage traditionnel. On ne stocke plus les données en ligne comme dans une table de SGBDR ou dans un fichier plat. Cette fois-ci, les documents sont indexés en fonction de leur contenu et d'après un vocabulaire prédéfini. Ainsi, chaque terme du vocabulaire devient une ligne et chaque document inséré se retrouve référencé dans les lignes correspondantes aux termes que le document contient. Derrière ce référencement se cache une base de données clé-valeur qui permet de sélectionner le document très rapidement une fois que la clé est trouvée.

Cette évolution dans la complexité de la base permet d'optimiser considérablement la recherche d'un document d'après son contenu. Cependant, pour que les recherches soient efficaces, il est nécessaire que le vocabulaire prédéfini soit assez large pour englober tous les termes que pourraient contenir les requêtes. Cet aspect est pris en charge par la base car le vocabulaire gonfle à chaque nouveau terme trouvé dans les documents insérés.

La recherche plein texte devient très efficace mais cela dépend des autres opérations. Pour chaque nouvelle insertion, l'indexation est à mettre à jour pour tous les termes qu'elle contient, ce qui peut être une lourde tâche pour un gros document ou lorsque le vocabulaire est devenu très volumineux ; pour chaque modification d'un document, cela demande à supprimer l'indexation en cours pour ce document et à en créer une nouvelle.

https://en.wikipedia.org/wiki/Column-oriented_DBMS

Les bases de données orientées graphes

On compte dans cette catégorie les serveurs Allegro, Neo4J et InfiniGraph. Ce type de base de données NoSQL est basé sur le concept clé-valeurs. Leur conception se veut plus orientée sur les relations entre les données. La subtilité qui différencie les SGBDR de ces systèmes réside dans le fait que les données ne contiennent pas les relations. Une collection est dédiée à lier les documents entre eux. Pour faire le rapprochement avec la théorie des graphes, on peut considérer que chaque document est un nœud et que chaque relation est un lien.

Ces bases semblent donc adaptées à des systèmes d'informations qui étaient prédisposés à persister leurs données dans des bases relationnelles. Elles apportent d'ailleurs une grosse optimisation dans la navigation par relation car il suffit au gestionnaire de suivre les liens là où un SGBDR aurait réalisé de coûteuses opérations de jointure.

https://en.wikipedia.org/wiki/Graph_database

https://fr.wikipedia.org/wiki/Base_de_donn%C3%A9es_orient%C3%A9e_graphe

Le choix en fonction du format des données

Les bases de données orientées document ont la particularité d'être fortement liées au format qu'elles supportent. Parmi les formats les plus courants, on trouve le XML et le JSON. Le format des documents que l'on souhaite persister sera déterminant sur le choix de la base de données documentaire. Si le format supporté par la base n'est pas adapté, les requêtes ne pourront pas concerner des paramètres internes aux documents. Beaucoup de bases supportent ces deux formats mais ce n'est pas

systematique. D'autres offrent la possibilité de persister des formats plus exotiques tels que des structures C++ ou des formats moins structurés comme du texte ou des sources binaires.

Une technologie en pleine effervescence

Les bases de données NoSQL sont des technologies récentes. Nous venons de voir que leur évolution a pris des chemins variés et il serait fort probable que ces systèmes nous réservent encore de quelques nouvelles orientations ces prochaines années. En épluchant les articles de magasins spécialisés, on peut se rendre compte de plusieurs lignes directrices dans la volonté de progression de cette technologie.

Les leaders du relationnel se lancent dans le NoSQL

Précédemment, nous avons constaté que beaucoup d'éditeurs se sont lancés dans le développement de bases de données NoSQL. Parmi ces éditeurs, on retrouve les grands noms du monde SGBDR tel qu'Oracle, IBM et Microsoft. Mais chacun a sa propre manière d'entrer sur ce nouveau marché.

Certains proposent de nouveaux produits

Oracle est entré sur le marché du NoSQL en fin 2014 avec « Oracle NoSQL Database ». Il s'agit d'une base de données clé-valeur qui offre tous les avantages que l'on associe aux systèmes NoSQL, à savoir haute disponibilité, scalabilité et distribution.

<http://www.oracle.com/fr/products/database/nosql/overview/index.html>

Microsoft tente aussi l'expérience avec « Microsoft Azure – DocumentDB ». A la différence d'Oracle, il ne s'agit pas d'un serveur que l'on peut déployer soi-même mais d'une offre en ligne d'hébergement de données. Il est donc difficile de savoir ce qui se cache derrière cette appellation. Sur leur page de présentation, il semblerait s'agir d'une base de données orientée documents.

<https://azure.microsoft.com/fr-fr/documentation/articles/documentdb-introduction/>

D'autres tentent la fusion des deux concepts

Relationnel ou base documentaire. Pourquoi choisir ? Pour aborder ce nouveau marché, certains éditeurs ont préféré adapter leur SGBDR plutôt que de repartir d'une page blanche.

Ainsi, IBM a fait évoluer sa base DB2 en lui ajoutant une fonctionnalité « supportant » le NoSQL. Cette idée semble difficile à comprendre car le concept NoSQL est apparu pour s'abstraire de la lourdeur des SGBDR. Alors comment peut-on annoncer qu'une base de données relationnelle est adaptée au NoSQL ? La réponse est dans l'appellation. NoSQL signifie Not Only SQL et n'implique en rien que la base portera les contraintes CAP. IBM offre donc la possibilité de stocker des documents comme un système NoSQL mais dans une base relationnelle. N'oublions pas que la manipulation de documents est en partie responsable de l'abandon des contraintes ACID du fait de leur structures diverses. Nous avons donc là un produit qui offre l'avantage de n'utiliser qu'une base de données pour manipuler des données relationnelles et des données documentaires mais qui, pour la peine, ne possède les avantages ni de l'un, ni de l'autre.

<http://www-01.ibm.com/software/data/db2/linux-unix-windows/nosql-support.html>

PostgreSQL s'est lancé dans le même chantier en permettant à sa dernière version de supporter les données documentaires. Celles-ci peuvent être stockées selon le modèle des bases de données orientées documents, ou selon le modèle clé-valeur mais contrairement à DB2 qui est adapté aux formats XML (pure ou décrit) et JSON, PostgreSQL ne supporte que des documents JSON.

http://info.enterprisedb.com/rs/enterprisedb/images/EDB_White_Paper_Using_the_NoSQL_Features_in_Postgres.pdf

Ces solutions hybrides peuvent paraître attrayantes à premier abord car elles permettent de répondre aux problématiques de stockages de documents et de données relationnelles en un seul et même produit. Cependant, même si ces systèmes offrent la fonctionnalité dite NoSQL par le stockage de document, ils n'en ont pas les idéologies : ils n'annoncent pas de gain de performance, ils ne sont pas plus scalable et ne sont pas plus adaptés à la haute disponibilité. Il paraît donc évident que ces produits ont été développés pour assurer une présence sur ce nouveau marché plus que pour offrir une vraie solution aux problématiques du BigData.

Tout n'est pas mauvais dans les SGBDR

Les bases de données relationnelles ont une suprématie évidente sur le stockage des données depuis les années 70. Il ne va pas être facile de bousculer les habitudes des développeurs qui ont utilisé ces systèmes depuis de nombreuses années. Les systèmes NoSQL ont au moins deux grosses lacunes comparé aux SGBDR qui font de ces nouveaux gestionnaires des produits peu attractifs : le langage de requête haut niveau et la garantie de l'intégrité des données. Les choses évoluent pour estomper ces manques.

SQL : Un langage qui a de l'avenir

Sur des bases de données comme MongoDB, on peut réaliser aisément des requêtes en filtrant les documents sur certaines propriétés qui lui sont propres. Lorsque les propriétés que l'on souhaite filtrer n'appartiennent plus au document, les opérations deviennent nettement plus compliquées. On peut éventuellement passer par des actions de MapReduce mais on ressent vite les limites du langage. Une opération de jointure en SQL s'écrit en une dizaine de mots alors qu'il faudrait plusieurs lignes pour la coder en MapReduce. Cet écart se réduit considérablement car certains éditeurs ont réussi à faire du langage SQL un langage de haut niveau pour les systèmes NoSQL.

C'est le cas avec CouchDB et le langage N1QL. Ce langage extrêmement proche du SQL permet d'interroger la base documentaire en offrant des fonctionnalités comme la jointure qui étaient très utilisées en SQL mais très difficile à réaliser sur un langage d'interrogation de base documentaire.

<http://docs.couchbase.com/4.0/n1ql/index.html>

Oracle propose également un outil nommé « Oracle Big Data SQL ». Celui-ci offre la possibilité de se brancher sur une base NoSQL telle que Hadoop et de l'interroger avec le langage SQL à l'image de ce qui est exécuté sur les bases de données Oracle.

<http://www.oracle.com/us/products/database/big-data-sql/overview/index.html>

D'une manière moins pratique, MongoDB a aussi réalisé un parallèle entre SQL et son langage de requête via une documentation qui lie les opérations SQL et les requêtes qu'il faudra exécuter pour réaliser les mêmes opérations. Ce détail se limite malheureusement aux actions simples de CRUD (Create, Read, Update, Delete).

<http://docs.mongodb.org/manual/reference/sql-comparison/>

Séduire les partisans des propriétés ACID

L'intégrité des données est une contrainte très importante dans de nombreux projets informatiques. Les mécanismes permettant d'assurer ces contraintes dans les SGBDR, aussi lourds soient-ils, sont des atouts indéniables pour le bon fonctionnement de ces projets. « Passer au NoSQL » est-il synonyme de « perdre les transactions, les schémas, les verrous, les contraintes d'intégrités... » ? La réponse n'est pas simple et ne peut être prononcée fermement par un oui ou un non.

Selon les systèmes, il sera possible de trouver des astuces ou des frameworks permettant d'adapter les propriétés ACID aux bases NoSQL. MongoDB en est un bon exemple. Pour réaliser des insertions de manière atomique, nous pouvons agréger toutes les opérations dans une seule requête d'insertion. On parvient ainsi à créer un semblant de transaction. Pour la réalisation de schémas ou de contraintes d'intégrités, nous avons pu découvrir dans le projet qu'il était possible de passer par le framework Mongoose. En revanche, aucun outil ne nous permet aujourd'hui d'effectuer simplement des opérations de rollback ou de lock. Si le besoin s'en fait sentir, le développeur sera contraint de réaliser sa propre librairie pour réaliser ces mécanismes. Ils seront certainement moins performant que s'ils étaient implémentés nativement dans la base de données mais il faut garder à l'esprit que cette solution existe. Dans le cadre de certains développements, cette option peut même être perçue comme un avantage car il sera alors possible de choisir quand appliquer ces contraintes.

Etant donné la volonté apparente de contenter les ex-utilisateurs de systèmes relationnels, je fais le pari que nous verrons prochainement de plus en plus d'éditeurs proposer, sur des bases de données NoSQL, des adaptations ou frameworks offrant les fonctionnalités requises par les propriétés ACID.

Le BigData pour tous sur le Cloud

NoSQL est annoncé comme la réponse à la haute volumétrie de données. Pour séduire encore plus de clients et les aider à quitter leurs bases de données relationnelles, plusieurs sites proposent d'héberger et de prendre en charge ces données. Le BigData n'est plus retreint aux grosses entreprises. Il est désormais accessible à n'importe quelle société sans besoins de déployer et de maintenir de nombreux serveurs. Ce type d'offres n'est pas né avec les bases de données documentaires mais elles constituent un véritable atout supplémentaire en faveur du NoSQL.

Pour plus de renseignements sur les différentes offres actuelles d'Amazon, Google et Microsoft, consultez ce comparatif qui est très intéressant :

<http://www.journaldunet.com/solutions/cloud-computing/comparatif-des-services-cloud-hadoop/>

Des avis partagés

Nous avons pu comparer les différents aspects qui forment les atouts de chaque système de gestion de base de données. Le monde NoSQL semble truffé de diverses optimisations le rendant très attractif mais les SGBDR ont toujours quelques cartes à jouer : celles de la maturité, ou celle de l'intégrité des données. Face à ces écarts qui ne s'estomperont peut être jamais, le choix est difficile dans la réalisation d'un nouveau projet. Les avis sont d'ailleurs très partagés.

NoSQL futur leader

L'article dont le lien se trouve ci-dessous transcrit une interview de Yann Aubry, travaillant pour MongoDB. Sa vision des systèmes NoSQL est engageante et il ne fait aucun doute que ces systèmes prendront prochainement le pas sur les systèmes relationnels.

Selon lui, la puissance et l'agilité des bases de données NoSQL leur permettent de s'imposer dans le cadre d'application web. On peut noter ici la restriction liée au type de projet. Les projets web ont la particularité d'être majoritairement axés sur la recherche d'informations contrairement aux applications client lourd qui sont plus adaptées aux applications de gestion, projets pour lesquels l'intégrité des données est très importante.

Il affirme également que *les bases NoSQL peuvent autant compléter les bases relationnelles que les remplacer*. Suite à cette étude, je suis aussi de cet avis mais, selon le type de projet, il ne faudra pas négliger les risques liés à la perte des contraintes ACID.

Le manque de structure des documents en NoSQL permettrait une conduite du changement plus simple dans le cadre de développement agile. Il est vrai qu'il est très périlleux de réaliser des évolutions rapides dans un projet dont les données sont fortement structurées. Il faut revoir la conception de la base, corriger les requêtes et le mapping pour enfin arriver à réaliser les modifications fonctionnelles attendues. En travaillant avec des données non structurées, la protection vis-à-vis du mauvais format est une contrainte que l'on prend en compte systématiquement, réduisant ainsi l'impact des futures évolutions de format.

Les systèmes d'enregistrements laissent place aux systèmes d'engagements, favorisant ainsi les technologies NoSQL. Il s'agit encore d'un aveu démontrant que les SGBDR et les systèmes NoSQL ont leur domaine de prédilection en termes de type de projet informatique.

<http://www.silicon.fr/base-donnees-nosql-impose-sgbd-93305.html>

L'expérience difficile du NoSQL

Ce second article est construit sur les impressions de Matthew Membrea, fondateur d'une entreprise spécialisée dans le développement logiciel et les services informatiques. Son expérience des systèmes NoSQL est nettement moins idyllique.

Le premier point noir relevé est lié au *nombre important de produits annoncés comme étant NoSQL*. Comme l'a démontré cette étude, même en isolant les distributions les plus sérieuses, on conserve une grande diversité dans les types de bases qui, pour chacun d'eux, correspondent à des optimisations complètement différentes. Les bases orientées documents favorisent l'insertion, les bases clé-valeur améliorent la sélection, les bases orientées colonnes sont à privilégier pour la recherche plein texte... Le passage au NoSQL s'accompagne donc de l'appréhension liée à la

découverte de cette nouvelle technologie et n'est pas simplifiée lorsque l'on doit s'en remettre au choix de l'organisation des données.

La perte des principes ACID force à les implémenter au cours des développements. Tant que les frameworks d'accès aux données NoSQL ne sont pas assez murs, ces développements seront incontournables pour garantir le bon fonctionnement des applications.

Les performances promises en valent-elle le détour ? C'est le sujet de cette étude et il semblerait que nous arriverons à la même conclusion que celle de cet article.

<http://www.developpez.com/actu/73650/-Pourquoi-je-ne-suis-pas-adepte-du-NoSQL-Un-professionnel-justifie-ce-choix-sur-la-base-de-plusieurs-criteres/>

Conclusion

Les systèmes NoSQL sont des technologies récentes aux performances très prometteuses.

Elles sont une alternative efficace face à la perte de performance des bases de données relationnelles dans le cadre d'une montée en charge. D'ailleurs, si votre système d'information est amené à traiter des volumes de données que l'on pourrait classer dans la catégorie du BigData, il est fort probable que vous n'ayez d'autre choix que de passer à une base NoSQL.

Pour tout autre projet de moins grande ampleur, il est important de connaître le type de données que vous devrez manipuler.

Si vos données ont un besoin évident en atomicité (ex : applications bancaires), en cohérence (ex : gestion de stocks), en isolation ou en durabilité, n'optez pas pour un système NoSQL qui ne saura vous garantir ces propriétés sur vos données.

Ensuite, il faut s'assurer que l'on sera prêt à s'affranchir des mécanismes tels que les transactions, les verrous ou les contraintes d'intégrité. Eventuellement, il faudra développer les fonctionnalités dont le manque aura le plus d'impact sur le bon fonctionnement de l'application.

En ayant accepté tous ces compromis, une dernière question se pose avant de s'orienter sur le NoSQL : le risque et le coût de la découverte de ses systèmes sera-t-il un bon investissement vis-à-vis des performances gagnées.

Maintenant que nous avons la conviction que le gain en performance ou le format varié des données justifie le passage au NoSQL, il faut choisir le système adapté au projet. Le premier élément déterminant est le type d'optimisation que l'on estime comme étant le plus important. Selon qu'il faille optimiser les insertions, les sélections, la navigation relationnelle ou les recherches plein texte, toutes les bases ne seront pas adaptées.

Ensuite, la bascule peut être opérée.

Le choix du passage au NoSQL est donc loin d'être évident. La technologie est peu mature, l'objectif d'optimisation est louable mais les solutions sont trop diverses et cela s'opère au dépend de contraintes sur lesquels nous avons l'habitude de nous reposer. Cela dit, le gain en performance est non négligeable, quel que soit le type d'optimisation ciblé. L'avenir du NoSQL est encourageant mais il semble ambitieux et risqué de miser aujourd'hui sur ces systèmes. Les architectures mixtes peuvent éventuellement être une bonne option en séparant les données régulièrement modifiées dans un SGBDR et les données transformées et adaptées à des recherches ou des analyses dans des systèmes NoSQL.